Lumentra Deployment Guide

Lumentra AI LLC

March 2026

Contents

Lumentra Production Deployment Guide **3**
Table of Contents . . . 4
1. Architecture Overview . . . 4
1.1 System Diagram . . . 4
1.2 Service Inventory . . . 5
1.3 Data Flow for an Inbound Call . . . 5
2. Prerequisites . . . 5
2.1 Infrastructure Requirements . . . 5
2.2 External Service Accounts . . . 6
2.3 Domain and DNS . . . 6
2.4 Local Tools . . . 6
3. Complete Environment Variable Reference . . . 6
3.1 Database (Supabase) . . . 6
3.2 LLM Configuration . . . 7
3.3 Voice Stack . . . 7
3.4 Telephony (SignalWire) . . . 7
3.5 SMS (Twilio) . . . 7
3.6 LiveKit . . . 7
3.7 Internal API Communication . . . 8
3.8 Server Configuration . . . 8
3.9 Dashboard Configuration . . . 8
3.10 Stripe (Billing - Optional) . . . 8
3.11 Infrastructure Stack . . . 8
4. Docker Deep-Dive . . . 9
4.1 Main docker-compose.yml . . . 9
4.2 API Dockerfile . . . 13
4.3 Dashboard Dockerfile . . . 13
4.4 Agent Dockerfile . . . 14
4.5 Infrastructure docker-compose . . . 14
4.6 Local Development docker-compose . . . 15
4.7 Network Architecture . . . 15
5. Server Provisioning and Hardening . . . 16
5.1 Hetzner Server Creation . . . 16
5.2 Server Hardening Script . . . 16
5.3 SSH Key Management . . . 18
5.4 Hetzner Cloud Firewall . . . 18
5.5 Enable SSH on Boot . . . 18
6. Coolify Deployment . . . 18
6.1 Installing Coolify . . . 18
6.2 Coolify Initial Setup . . . 19
6.3 Deploy the API . . . 19
6.4 Deploy the Dashboard . . . 19
6.5 The Artisan Tinker Workaround . . . 19

6.6 Auto-Deploy via Webhooks . . . 20
6.7 Coolify Network Architecture . . . 20
7. LiveKit Voice Stack Deployment . . . 20
7.1 Overview . . . 20
7.2 LiveKit Server Configuration . . . 20
7.3 SIP Configuration . . . 21
7.4 SIP Trunk and Dispatch Rules . . . 22
7.5 Production Deployment on Server . . . 23
7.6 Voice Agent Pipeline Details . . . 23
8. SIP Trunk Setup (SignalWire) . . . 24
8.1 SignalWire Account Creation . . . 24
8.2 Phone Number Purchase . . . 25
8.3 Configure SIP Domain App . . . 25
8.4 Configure the Phone Number for SIP . . . 25
8.5 Create LiveKit SIP Trunk . . . 25
8.6 Create Dispatch Rule . . . 25
8.7 Testing . . . 26
8.8 Troubleshooting SIP . . . 26
9. Nginx Configuration . . . 26
9.1 Development Configuration . . . 26
9.2 Production Configuration . . . 27
9.3 Installing and Configuring Nginx . . . 29
10. SSL/TLS Configuration . . . 29
10.1 Certbot Installation . . . 29
10.2 Obtaining Certificates . . . 29
10.3 Auto-Renewal . . . 29
10.4 Nginx SSL Integration . . . 30
10.5 Cloudflare SSL Mode . . . 30
10.6 Cloudflare Origin Certificates (Alternative) . . . 30
11. DNS Configuration . . . 30
11.1 Add Domain to Cloudflare . . . 30
11.2 DNS Records . . . 30
11.3 Cloudflare Settings . . . 31
12. Database Setup and Migrations . . . 31
12.1 Supabase Project Setup . . . 31
12.2 Migration Reference . . . 31
12.3 Running Migrations . . . 33
12.4 Migration Ordering Rules . . . 33
12.5 Rollback Procedures . . . 34
12.6 Writing New Migrations . . . 34
12.7 Schema Files . . . 34
13. Post-Deployment Verification . . . 34
13.1 Health Check Endpoints . . . 34
13.2 Verification Checklist . . . 35
13.3 Testing a Phone Call End-to-End . . . 35
14. Monitoring and Observability . . . 35
14.1 Health Check Monitoring . . . 35
14.2 Uptime Kuma Setup . . . 36
14.3 Log Monitoring . . . 36
14.4 System Resource Monitoring . . . 37
14.5 Voice Call Metrics . . . 37
14.6 Database Monitoring . . . 37
15. Backup and Recovery . . . 38
15.1 Backup Script Setup . . . 38
15.2 Backup Schedule . . . 38
15.3 Backup Retention . . . 38
15.4 Manual Backup . . . 38

15.5 Backup Command Details . . . 38
15.6 Restoring from Backup . . . 39
15.7 Disaster Recovery Plan . . . 39
15.8 File Backup . . . 40
16. Update Procedures . . . 40
16.1 Updating the API . . . 40
16.2 Updating the Dashboard . . . 40
16.3 Updating the Voice Agent . . . 40
16.4 Updating LiveKit Server . . . 41
16.5 Updating LiveKit SIP Bridge . . . 41
16.6 Database Migrations During Updates . . . 41
16.7 Zero-Downtime Considerations . . . 42
17. Scaling Guide . . . 42
17.1 Current Capacity (Single CCX13) . . . 42
17.2 Horizontal Scaling: API . . . 42
17.3 Scaling: Voice Agent . . . 43
17.4 Scaling: LiveKit Server . . . 43
17.5 Database Connection Pooling . . . 43
17.6 Scaling: Redis . . . 43
17.7 Scaling Thresholds . . . 43
18. Troubleshooting . . . 44
18.1 API Issues . . . 44
18.2 Dashboard Issues . . . 44
18.3 Voice Call Issues . . . 44
18.4 SIP Issues . . . 45
18.5 SSL Issues . . . 45
18.6 Coolify Issues . . . 46
18.7 Database Issues . . . 46
18.8 Docker Issues . . . 46
19. Security Checklist . . . 47
19.1 Server Security . . . 47
19.2 Network Security . . . 47
19.3 Application Security . . . 47
19.4 Credentials Security . . . 47
19.5 Generating Secure Secrets . . . 48
20. Cost Estimates . . . 48
20.1 Monthly Infrastructure Costs . . . 48
20.2 Per-Call Voice Costs . . . 48
20.3 Monthly Volume Estimates . . . 48
21. Quick Command Reference . . . 49
21.1 SSH Access . . . 49
21.2 Container Management . . . 49
21.3 LiveKit Stack . . . 49
21.4 LiveKit CLI . . . 49
21.5 Backup and Database . . . 50
21.6 Coolify Deployment . . . 50
21.7 SSL and Nginx . . . 50
21.8 System Monitoring . . . 50
21.9 Hetzner Cloud CLI . . . 51

Lumentra Production Deployment Guide

Complete reference for deploying the Lumentra multi-tenant voice AI platform in production. Covers every service, configuration option, and operational procedure: the Node.js API, Next.js dashboard, LiveKit-based voice pipeline (LiveKit Server, SIP bridge, Python voice agent), Nginx reverse proxy, SSL/TLS, database migrations, backups, monitoring, scaling, and troubleshooting.

Table of Contents

1. Architecture Overview
2. Prerequisites
3. Complete Environment Variable Reference
4. Docker Deep-Dive
5. Server Provisioning and Hardening
6. Coolify Deployment
7. LiveKit Voice Stack Deployment
8. SIP Trunk Setup (SignalWire)
9. Nginx Configuration
10. SSL/TLS Configuration
11. DNS Configuration
12. Database Setup and Migrations
13. Post-Deployment Verification
14. Monitoring and Observability
15. Backup and Recovery
16. Update Procedures
17. Scaling Guide
18. Troubleshooting
19. Security Checklist
20. Cost Estimates
21. Quick Command Reference

---

1. Architecture Overview

1.1 System Diagram

```
                          Internet
                             |
               +-------------+------------------+
               |                                |
         HTTPS traffic                    SIP (UDP 5060)
               |                                |
         +-----v--------+         +-------------v-------------+
         |    Nginx      |         | LiveKit SIP Bridge        |
         |  (SSL/TLS)    |         | (livekit/sip:v1.8)        |
         |  Port 80/443  |         | host network, port 5060   |
         +-----+---------+         | RTP: 10000-20000          |
               |                   +-------------+-------------+
      +--------+--------+                        |
      |                 |                         |
+-----v------+   +------v-------+    +-----------v-----------+
| Lumentra   |   | Lumentra     |    | LiveKit Server        |
| API        |   | Dashboard    |    | (livekit/livekit:v1.8) |
| Node.js    |   | Next.js 16   |    | host network          |
| Port 3100  |   | Port 3000    |    | API: 7880             |
+-----+------+   +--------------+    | TCP: 7881             |
      |                               | ICE: 50000-60000      |
      +--------+--------+            +-----------+-----------+
               |                                |
      +--------v--------+           +-----------v-----------+
      | Supabase         |           | LiveKit Python Agent  |
```

```
| PostgreSQL       |           | (lumentra-agent)      |
| (managed cloud)  |           | STT: Deepgram nova-3  |
+------------------+           | LLM: GPT-4.1-mini     |
                               | TTS: Cartesia Sonic-3  |
         +--------+           +-----------+-----------+
         | Redis  |                       |
         | 6379   |<----------------------+
         +--------+           (room coordination)
```

1.2 Service Inventory

Service	Technology	Port(s)	Net Mode	Managed By
API	Node.js / Hono / TS	3100	Bridge	Coolify
Dashboard	Next.js 16 / React	3000	Bridge	Coolify
LiveKit Server	livekit-server:v1.8	7880, 7881, 50k+	Host	docker-compose
LiveKit SIP	livekit/sip:v1.8	5060, 10k-20k	Host	docker-compose
Voice Agent	Python 3.12 / LK SDK	(internal)	Bridge	docker-compose
Redis	Redis 7 Alpine	6379 (localhost)	Bridge	docker-compose
Nginx	nginx:alpine	80, 443	Bridge	System/Coolify
PostgreSQL	Supabase (managed)	5432 (remote)	N/A	Supabase

1.3 Data Flow for an Inbound Call

1. Caller dials the business phone number (e.g., +19458001233).
2. SignalWire receives the call and sends a SIP INVITE to the LiveKit SIP bridge on port 5060.
3. LiveKit SIP bridge creates a LiveKit room and bridges the SIP call audio into it.
4. The dispatch rule matches the dialed number and routes to the `lumentra-voice-agent` agent.
5. The Python agent receives the job, connects to the room, and reads the SIP attributes (dialed number, caller phone).
6. The agent calls `GET /internal/tenants/by-phone/:phone` on the Lumentra API to fetch tenant config and system prompt.
7. The agent session starts with Deepgram STT, OpenAI GPT-4.1-mini LLM, and Cartesia TTS.
8. During the conversation, the agent may call tools (`check_availability`, `create_booking`, `transfer_to_human`, etc.) via `POST /internal/voice-tools/:action`.
9. When the call ends, the agent posts call data to `POST /internal/calls/log`.
10. The API auto-creates a CRM contact and runs post-call automation (deals, tasks).

---

2. Prerequisites

2.1 Infrastructure Requirements

Requirement	Minimum	Recommended
Server	2 vCPU, 4 GB RAM	4 vCPU, 8 GB RAM
Disk	40 GB SSD	80 GB SSD
OS	Ubuntu 22.04 LTS	Ubuntu 24.04 LTS
Location	US East	Ashburn, VA (ash-dc1)
Provider	Any VPS	Hetzner Cloud CCX13

2.2 External Service Accounts

Service	Purpose	URL	Required
Supabase	PostgreSQL + Auth	https://supabase.com	Yes
SignalWire	SIP trunk + phone numbers	https://signalwire.com	Yes
Deepgram	Speech-to-text (STT)	https://console.deepgram.com	Yes
OpenAI	LLM (GPT-4.1-mini)	https://platform.openai.com	Yes
Cartesia	Text-to-speech (TTS)	https://play.cartesia.ai	Yes
Cloudflare	DNS + DDoS protection	https://dash.cloudflare.com	Yes
Gemini	Chat widget LLM (primary)	https://aistudio.google.com	Yes
Groq	LLM fallback	https://console.groq.com	Optional
Twilio	SMS notifications	https://twilio.com	Optional
Stripe	Billing	https://dashboard.stripe.com	Optional

2.3 Domain and DNS

- One domain purchased (e.g., `lumentraai.com`).
- DNS managed through Cloudflare (free tier).
- Required subdomains: `api.lumentraai.com`, `app.lumentraai.com`.

2.4 Local Tools

- SSH client with ed25519 key pair.
- `hcloud` CLI v1.60+ (Hetzner Cloud CLI). The apt version (v1.13) is too old; install from GitHub releases to `~/.local/bin/hcloud`.
- Docker and Docker Compose (for local testing).
- Node.js 22+ and npm (via nvm).
- Python 3.12+ (for agent local testing).

---

3. Complete Environment Variable Reference

This section documents every environment variable across all services. Variables are sourced from the following `.env.example` files:

- `/.env.example` – Root (docker-compose.yml)
- `/lumentra-api/.env.example` – API service
- `/lumentra-dashboard/.env.example` – Dashboard service
- `/lumentra-agent/.env.example` – Voice agent
- `/infrastructure/.env.example` – Infrastructure stack

3.1 Database (Supabase)

- **`SUPABASE_URL`** (URL, required): Supabase project URL. Example: `https://xxxx.supabase.co`. Without it, API cannot connect to database.
- **`SUPABASE_ANON_KEY`** (String, required): Supabase anonymous (public) key. Without it, dashboard auth fails.
- **`SUPABASE_SERVICE_ROLE_KEY`** (String, required): Supabase service role key (bypasses RLS). Without it, all API DB operations fail.
- **`DATABASE_URL`** (URL, optional): Direct PostgreSQL connection string. Only needed for direct pg_dump/migrations.

Security note: The `SUPABASE_SERVICE_ROLE_KEY` has full database access and must never be exposed to the client. Only the API service should use it.

3.2 LLM Configuration

The API uses a multi-provider fallback chain for chat widget responses: Gemini (primary) -> OpenAI -> Groq. The voice agent uses OpenAI GPT-4.1-mini exclusively.

- **GEMINI_API_KEY** (String, required): Google Gemini API key. Without it, chat widget primary LLM fails.
- **GEMINI_MODEL** (String, optional): Gemini model. Default: `gemini-2.5-flash`.
- **OPENAI_API_KEY** (String, required): OpenAI API key. Without it, voice agent and chat fallback fail.
- **OPENAI_MODEL** (String, optional): OpenAI model for chat. Default: `gpt-4.1-mini`.
- **GROQ_API_KEY** (String, optional): Groq API key. Without it, third-level chat fallback unavailable.
- **GROQ_CHAT_MODEL** (String, optional): Groq chat model. Default: `llama-3.1-8b-instant`.
- **GROQ_TOOL_MODEL** (String, optional): Groq tool-calling model. Default: `llama-3.3-70b-versatile`.
- **VOICE_LLM_PROVIDER_ORDER** (String, optional): Comma-separated voice LLM fallback order. Default: `openai,groq,gemini`.
- **LLM_PROVIDER** (String, optional): Primary LLM provider. Default: `groq`.
- **TOGETHER_API_KEY** (String, optional): Together AI key for fine-tuned models.

3.3 Voice Stack

- **DEEPGRAM_API_KEY** (String, required): Deepgram API key for STT. Without it, voice agent cannot transcribe speech.
- **CARTESIA_API_KEY** (String, required): Cartesia API key for TTS. Without it, voice agent cannot generate speech.
- **VOICE_PROVIDER** (String, optional): Voice pipeline provider. Default: `custom`.
- **DEEPGRAM_MODEL** (String, optional): Deepgram STT model. Default: `nova-2-phonecall`.

3.4 Telephony (SignalWire)

- **SIGNALWIRE_PROJECT_ID** (UUID, required): SignalWire project UUID. Without it, cannot manage phone numbers.
- **SIGNALWIRE_API_TOKEN** (String, required): SignalWire API auth token. Without it, cannot manage phone numbers.
- **SIGNALWIRE_SPACE_URL** (URL, required): SignalWire space domain. Example: `yourspace.signalwire.com`.
- **SIGNALWIRE_PHONE_NUMBER** (E.164, optional): Phone number for outbound calls/SMS.
- **SIGNALWIRE_WEBHOOK_SECRET** (String, required): Webhook signature verification secret (32+ random chars).
- **STREAM_SIGNING_SECRET** (String, required): Stream signing secret for media (32+ random chars).

3.5 SMS (Twilio)

- **TWILIO_ACCOUNT_SID** (String, optional): Twilio account SID. Without it, SMS features unavailable.
- **TWILIO_AUTH_TOKEN** (String, optional): Twilio auth token. Without it, SMS features unavailable.
- **TWILIO_PHONE_NUMBER** (E.164, optional): Twilio sender phone number. Example: +18559147321.

3.6 LiveKit

- **LIVEKIT_API_KEY** (String, required): LiveKit API key (must match livekit.yaml). Default: devkey.
- **LIVEKIT_API_SECRET** (String, required): LiveKit API secret (must match livekit.yaml). Default: `secret`.
- **LIVEKIT_URL** (URL, required): LiveKit server WebSocket URL. Default: `ws://localhost:7880`.

3.7 Internal API Communication

- **INTERNAL_API_KEY** (String, required): Shared secret for agent-to-API auth (64+ random chars). Without it, agent cannot fetch tenant config or log calls.
- **INTERNAL_API_URL** (URL, required): URL where agent reaches the API. Default: `http://localhost:3100`. Production: `http://10.0.1.5:3100`.

Critical: The `INTERNAL_API_KEY` must be identical in both the API service and the agent service. A mismatch causes 403 errors on all internal endpoints.

3.8 Server Configuration

- **PORT** (Number, optional): API server port. Default: `3100`.
- **NODE_ENV** (String, optional): Node.js environment. Default: `development`.
- **BACKEND_URL** (URL, required): Public-facing API URL. Example: `https://api.lumentraai.com`. Without it, webhook URLs are incorrect.
- **FRONTEND_URL** (URL, optional): Public-facing dashboard URL. Default: `http://localhost:3000`. Without it, CORS and redirect issues.
- **ENCRYPTION_KEY** (String, required): Key for encrypting data at rest (32+ random chars).
- **VAPI_WEBHOOK_SECRET** (String, optional): Legacy Vapi webhook secret (pre-LiveKit).

3.9 Dashboard Configuration

- **NEXT_PUBLIC_API_URL** (URL, required): API URL, baked into client JS at build time. Default: `http://localhost:3100`.
- **NEXT_PUBLIC_APP_URL** (URL, optional): Dashboard URL for redirects. Default: `http://localhost:3000`.
- **NEXT_PUBLIC_AUTH_REDIRECT_URL** (URL, optional): Supabase auth callback URL. Default: `http://localhost:3000/auth/callback`.
- **NEXT_PUBLIC_SUPABASE_URL** (URL, required): Supabase URL (client-side). Without it, auth and queries fail.
- **NEXT_PUBLIC_SUPABASE_ANON_KEY** (String, required): Supabase anon key (client-side). Without it, auth and queries fail.
- **NEXT_PUBLIC_TENANT_ID** (UUID, optional): Default tenant ID for single-tenant dev mode. Default: `dev-tenant`.

Important: All `NEXT_PUBLIC_*` variables are embedded into the JavaScript bundle at build time. Changing them requires a full rebuild and redeploy of the dashboard.

3.10 Stripe (Billing - Optional)

- **NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY** (String, optional): Stripe publishable key. Without it, billing UI unavailable.
- **STRIPE_SECRET_KEY** (String, optional): Stripe secret key. Without it, billing API calls fail.
- **STRIPE_WEBHOOK_SECRET** (String, optional): Stripe webhook endpoint secret.
- **STRIPE_STARTER_PRICE_ID** (String, optional): Price ID for Starter plan.
- **STRIPE_PRO_PRICE_ID** (String, optional): Price ID for Professional plan.
- **STRIPE_ENTERPRISE_PRICE_ID** (String, optional): Price ID for Enterprise plan.

3.11 Infrastructure Stack

These variables are used by `/infrastructure/docker-compose.infrastructure.yml` for the self-hosted infrastructure stack (PostgreSQL, Authentik, MinIO).

- **POSTGRES_USER** (String, optional): PostgreSQL superuser name. Default: `lumentra`.
- **POSTGRES_PASSWORD** (String, required): PostgreSQL superuser password (32+ random chars). Without it, database cannot start.
- **POSTGRES_DB** (String, optional): Default database name. Default: `lumentra`.
- **AUTHENTIK_DB_NAME** (String, optional): Authentik database name. Default: `authentik`.

- **AUTHENTIK_SECRET_KEY** (String, required): Authentik secret key (64+ hex chars). Without it, Authentik cannot start.
- **SMTP_HOST** (String, optional): SMTP server host. Default: `smtp.resend.com`.
- **SMTP_PORT** (Number, optional): SMTP server port. Default: 587.
- **SMTP_USERNAME** (String, optional): SMTP username. Default: `resend`.
- **SMTP_PASSWORD** (String, required): SMTP password (Resend API key). Without it, email notifications fail.
- **SMTP_FROM** (String, optional): From address for emails. Default: `noreply@lumentra.ai`.
- **MINIO_ROOT_USER** (String, required): MinIO root username. Without it, MinIO cannot start.
- **MINIO_ROOT_PASSWORD** (String, required): MinIO root password (8+ chars). Without it, MinIO cannot start.
- **MINIO_BROWSER_REDIRECT_URL** (URL, optional): MinIO console redirect URL. Default: `https://storage.lum`

Note about Hetzner and SMTP: Hetzner blocks standard SMTP ports (25, 465). Use Resend on port 587 or another provider that supports non-standard SMTP ports.

---

4. Docker Deep-Dive

4.1 Main docker-compose.yml

File: `/docker-compose.yml`

This is the primary composition file that runs the full stack including API, Dashboard, Redis, LiveKit, SIP, Agent, and an optional Nginx reverse proxy.

```yaml
api:
  build:
    context: ./lumentra-api
    dockerfile: Dockerfile
  ports:
    - "3100:3100"
  restart: unless-stopped
  deploy:
    resources:
      limits:
        cpus: "2"
        memory: 2G
      reservations:
        cpus: "0.5"
        memory: 512M
  logging:
    driver: "json-file"
    options:
      max-size: "50m"
      max-file: "3"
  healthcheck:
    test: ["CMD", "wget", "-q", "--spider", "http://localhost:3100/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 15s
```

4.1.1 API Service

Property	Value	Purpose
Base image	`node:20-alpine`	Minimal Node.js runtime
Build strategy	Multi-stage (3 stages)	Final image ~120MB
Port	3100	HTTP API
CPU limit	2 cores	Prevents CPU starvation
Memory limit	2 GB	Prevents OOM from leaks
CPU reservation	0.5 cores	Minimum guaranteed CPU
Mem reservation	512 MB	Minimum guaranteed memory
Log max-size	50 MB per file	Prevents disk full
Log max-file	3 files	150 MB total logs
Health check	`GET /health`	200 if healthy, 503 if degraded
Health interval	30s	Checks every 30 seconds
Start period	15s	Grace period before checks
Restart policy	`unless-stopped`	Auto-restart, not on manual stop

```yaml
dashboard:
  build:
    context: ./lumentra-dashboard
    dockerfile: Dockerfile
    args:
      - NEXT_PUBLIC_API_URL=${NEXT_PUBLIC_API_URL:-http://api:3100}
  ports:
    - "3000:3000"
  depends_on:
    api:
      condition: service_healthy
  restart: unless-stopped
```

4.1.2 Dashboard Service

Property	Value	Purpose
Base image	`node:20-alpine`	Minimal Node.js runtime
Build strategy	Multi-stage, standalone	~80MB final image
Port	3000	HTTP (Next.js)
Build args	4 `NEXT_PUBLIC_*` vars	Embedded at build time
Depends on	`api` (service_healthy)	Waits for API health check
Restart policy	`unless-stopped`	Auto-restart on crash
User	`nextjs` (UID 1001)	Non-root for security

Important: The dashboard depends on the API with `condition: service_healthy`. If the API health check fails, the dashboard will not start.

```yaml
redis:
  image: redis:7-alpine
  ports:
    - "127.0.0.1:6379:6379"
  restart: unless-stopped
  volumes:
    - redis-data:/data
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
```

```
    interval: 10s
    timeout: 5s
    retries: 3
```

4.1.3 Redis Service

Property	Value	Purpose
Image	`redis:7-alpine`	Lightweight Redis 7
Port binding	`127.0.0.1:6379:6379`	Localhost only, not exposed
Volume	`redis-data:/data`	Persists AOF/RDB across restarts
Health check	`redis-cli ping`	PONG when healthy

Redis is used exclusively by LiveKit Server and SIP bridge for room coordination. It is not used by the API or dashboard.

```
livekit:
  image: livekit/livekit-server:v1.8
  network_mode: host
  volumes:
    - ./lumentra-agent/livekit/livekit.yaml:/etc/livekit.yaml:ro
  command: --config /etc/livekit.yaml
  depends_on:
    redis:
      condition: service_healthy
  restart: unless-stopped
```

4.1.4 LiveKit Server

Property	Value	Purpose
Image	`livekit/livekit-server:v1.8`	WebRTC SFU
Network mode	`host`	Required for ICE/UDP ranges
Config mount	Read-only bind mount	livekit.yaml from source
Depends on	Redis (healthy)	Needs Redis for room state
Log rotation	20 MB x 3 files	60 MB total retention

Why host networking? LiveKit needs direct access to the host's network stack for WebRTC ICE candidate negotiation. UDP ports 50000-60000 must be directly accessible from the internet.

```
sip:
  image: livekit/sip:v1.8
  network_mode: host
  environment:
    - SIP_CONFIG_FILE=/etc/sip.yaml
  volumes:
    - ./lumentra-agent/livekit/sip.yaml:/etc/sip.yaml:ro
  depends_on:
    redis:
      condition: service_healthy
  restart: unless-stopped
```

4.1.5 LiveKit SIP Bridge

Property	Value	Purpose
Image	`livekit/sip:v1.8`	SIP-to-WebRTC bridge
Network mode	`host`	Required for SIP/RTP UDP ports
Config	`sip.yaml` bind mount	SIP and Redis configuration
Depends on	Redis (healthy)	Room coordination via Redis

```yaml
agent:
  build:
    context: ./lumentra-agent
    dockerfile: Dockerfile
  environment:
    - LIVEKIT_URL=ws://172.17.0.1:7880
    - LIVEKIT_API_KEY=${LIVEKIT_API_KEY}
    - LIVEKIT_API_SECRET=${LIVEKIT_API_SECRET}
    - INTERNAL_API_URL=http://api:3100
    - INTERNAL_API_KEY=${INTERNAL_API_KEY}
    - DEEPGRAM_API_KEY=${DEEPGRAM_API_KEY}
    - OPENAI_API_KEY=${OPENAI_API_KEY}
    - CARTESIA_API_KEY=${CARTESIA_API_KEY}
  depends_on:
    api:
      condition: service_healthy
  restart: unless-stopped
  deploy:
    resources:
      limits:
        cpus: "2"
        memory: 2G
      reservations:
        cpus: "0.5"
        memory: 512M
```

4.1.6 Voice Agent

Property	Value	Purpose
Base image	`python:3.12-slim`	Python for LiveKit Agents SDK
Build step	Deps + turn detector	Model cached in image layer
CPU limit	2 cores	STT/TTS is CPU-intensive
Memory limit	2 GB	VAD + turn detector models
LiveKit URL	`ws://172.17.0.1:7880`	Docker bridge to host LiveKit
Internal API URL	`http://api:3100`	Docker service name resolution

Networking note: The agent uses Docker bridge networking (not host mode), so it reaches LiveKit via `172.17.0.1` (Docker's default gateway to the host). In Coolify deployments, the agent reaches the API via `http://10.0.1.5:3100` (Coolify's internal network IP).

```yaml
nginx:
  image: nginx:alpine
  ports:
```

```yaml
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - ./certs:/etc/nginx/certs:ro
  depends_on:
    - api
    - dashboard
  restart: unless-stopped
  profiles:
    - production
```

4.1.7 Nginx (Production Profile) The Nginx service is only started when using the `production` profile:

```
docker compose --profile production up -d
```

In Coolify deployments, Coolify's built-in Caddy/Traefik handles SSL termination instead, so this service is not used.

4.2 API Dockerfile

File: `/lumentra-api/Dockerfile`

Three-stage multi-stage build:

1. **deps** – Installs production-only dependencies (`npm ci --only=production`).
2. **builder** – Installs all dependencies, copies source, runs `npm run build` (esbuild).
3. **runner** – Copies production `node_modules` and compiled `dist/`, runs as non-root user `lumentra` (UID 1001).

```
Stage    | Contents                     | Size
---------|------------------------------|---------
deps     | production node_modules      | ~80 MB
builder  | full node_modules + dist     | ~200 MB
runner   | prod node_modules + dist     | ~120 MB
```

The `CACHE_BUST` build arg can be incremented to force a full rebuild when Docker layer caching is too aggressive.

4.3 Dashboard Dockerfile

File: `/lumentra-dashboard/Dockerfile`

Three-stage build optimized for Next.js standalone output:

1. **deps** – Installs all dependencies.
2. **builder** – Copies source, sets `NEXT_PUBLIC_*` build args as ENV, runs `npm run build`.
3. **runner** – Copies `public/`, `.next/standalone/`, `.next/static/`, runs as non-root user `nextjs` (UID 1001). Hostname bound to `0.0.0.0`.

Build-time args that must be set:

Arg	Default
`NEXT_PUBLIC_API_URL`	`https://api.lumentraai.com`
`NEXT_PUBLIC_APP_URL`	`https://app.lumentraai.com`
`NEXT_PUBLIC_SUPABASE_URL`	(no default – required)
`NEXT_PUBLIC_SUPABASE_ANON_KEY`	(no default – required)

4.4 Agent Dockerfile

File: `/lumentra-agent/Dockerfile`

Simple single-stage build:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY agent.py tools.py tenant_config.py call_logger.py api_client.py ./
RUN python agent.py download-files    # Pre-downloads turn detector model
CMD ["python", "agent.py", "start"]
```

Key dependencies (from `requirements.txt`):

Package	Version	Purpose
`livekit-agents`	>=1.4.3	Agent framework
`livekit-plugins-deepgram`	>=1.0.0	Deepgram STT integration
`livekit-plugins-cartesia`	>=1.0.0	Cartesia TTS integration
`livekit-plugins-openai`	>=1.0.0	OpenAI LLM integration
`livekit-plugins-silero`	>=1.0.0	Silero VAD (voice activity detect)
`livekit-plugins-noise-cancellation`	>=0.2.0	BVC telephony noise cancellation
`livekit-plugins-turn-detector`	>=1.0.0	Multilingual turn detection
`httpx`	>=0.27.0	Async HTTP client for API calls
`python-dotenv`	>=1.0.0	Environment variable loading

4.5 Infrastructure docker-compose

File: `/infrastructure/docker-compose.infrastructure.yml`

Defines self-hosted infrastructure services. Deployed separately via Coolify.

Service	Image	Ports	Memory
`postgres`	`postgres:16-alpine`	127.0.0.1:5432	1 GB
`authentik-server`	`goauthentik/server:2025.10`	9000, 9443	1 GB
`authentik-worker`	`goauthentik/server:2025.10`	(none)	768 MB
`minio`	`minio/minio:latest`	9002, 9003	512 MB
`minio-init`	`minio/mc:latest`	(none)	(none)
`authentik-db-init`	`postgres:16-alpine`	(none)	(none)

Volumes: `postgres` mounts `/mnt/volume-ash-1/postgresql`; `authentik-server` and `authentik-worker` use `authentik_media`, `authentik_templates`, and `authentik_certs`; `minio` mounts `/mnt/volume-ash-1/minio`. The `minio-init` and `authentik-db-init` services run once and exit.

PostgreSQL performance tuning flags (via command args):

Flag	Value	Purpose
`shared_buffers`	256 MB	Shared memory for caching
`effective_cache_size`	768 MB	OS cache estimate
`maintenance_work_mem`	64 MB	Memory for VACUUM, CREATE INDEX

Flag	Value	Purpose
`checkpoint_completion_target`	0.9	Spread checkpoint writes
`wal_buffers`	16 MB	WAL write buffering
`random_page_cost`	1.1	SSD-optimized (default is 4.0)
`effective_io_concurrency`	200	SSD-optimized concurrent I/O
`work_mem`	4 MB	Per-operation sort memory
`max_connections`	100	Max concurrent connections
`log_statement`	mod	Logs INSERT/UPDATE/DELETE
`log_min_duration_statement`	100 ms	Logs slow queries

Critical pre-deployment checklist for infrastructure:

1. Disable "Delete Unused Volumes" in Coolify Server Settings -> Docker Cleanup.
2. Mount Hetzner volume to `/mnt/volume-ash-1`.
3. Create directories: `mkdir -p /mnt/volume-ash-1/{postgresql,minio,backups}`.
4. Set permissions: `chown -R 999:999 /mnt/volume-ash-1/postgresql && chown -R 1000:1000 /mnt/volume-ash-1/minio`.

4.6 Local Development docker-compose

File: `/lumentra-api/docker-compose.local.yml`

Provides a local PostgreSQL for testing migrations without affecting Supabase:

```yaml
services:
  postgres:
    image: postgres:15-alpine
    container_name: lumentra-postgres-local
    ports:
      - "5434:5432"    # Port 5434 to avoid conflicts
    environment:
      POSTGRES_USER: lumentra
      POSTGRES_PASSWORD: localdev123456789
      POSTGRES_DB: lumentra
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./schema:/docker-entrypoint-initdb.d:ro
```

Start with: `docker compose -f docker-compose.local.yml up -d`

4.7 Network Architecture

```
lumentra-network (bridge, default)
  |-- api (3100)
  |-- dashboard (3000)
  |-- redis (6379, localhost only)
  |-- agent (internal)
  |-- nginx (80, 443) [production profile only]

host network
  |-- livekit (7880, 7881, 50000-60000)
  |-- sip (5060, 10000-20000)

lumentra-infrastructure (bridge, separate)
  |-- postgres (127.0.0.1:5432)
  |-- authentik-server (9000, 9443)
  |-- authentik-worker (internal)
```

```
|-- minio (9002, 9003)
```

---

5. Server Provisioning and Hardening

5.1 Hetzner Server Creation

1. Log into Hetzner Cloud Console.
2. Create a new server:
   - **Location:** Ashburn, VA (ash-dc1) – best for US-based SIP trunks.
   - **Image:** Ubuntu 24.04 LTS.
   - **Type:** CCX13 (4 vCPU AMD, 8 GB RAM, 80 GB SSD, 20 TB traffic).
   - **SSH Key:** Add your public key.
   - **Firewall:** Create `lumentra-voice` (see Section 5.4).
   - **Volume:** 10 GB attached at `/mnt/volume-ash-1` (for database and object storage).
3. Note the IPv4 address (e.g., `178.156.205.145`).

5.2 Server Hardening Script

File: `/hetzner/1-setup-server.sh`

Run as root immediately after server creation:

```
scp hetzner/1-setup-server.sh root@YOUR_IP:/root/
ssh root@YOUR_IP "chmod +x /root/1-setup-server.sh && bash /root/1-setup-server.sh"
```

The script performs the following actions:

```
apt update && apt upgrade -y
```

5.2.1 System Updates

5.2.2 Essential Tools Installation

Installs: `curl`, `wget`, `git`, `htop`, `vim`, `unzip`, `ufw`, `fail2ban`, `jq`.

5.2.3 Deploy User Creation

Creates a `deploy` user with: - Home directory at `/home/deploy`. - Added to `sudo` group. - Passwordless sudo: `deploy ALL=(ALL) NOPASSWD:ALL`. - SSH keys copied from root.

5.2.4 SSH Hardening

Creates `/etc/ssh/sshd_config.d/hardening.conf`:

Setting	Value	Purpose
`PermitRootLogin`	`prohibit-password`	Root can use keys only, no password
`PasswordAuthentication`	`no`	Keys only for all users
`PubkeyAuthentication`	`yes`	Enable public key auth
`PermitEmptyPasswords`	`no`	Block empty passwords
`X11Forwarding`	`no`	Disable X11 (not needed for server)
`MaxAuthTries`	`3`	Lock after 3 failed attempts
`ClientAliveInterval`	`300`	Ping idle clients every 5 min
`ClientAliveCountMax`	`2`	Disconnect after 2 missed pings
`Protocol`	`2`	SSH protocol 2 only
`AllowUsers`	`deploy root`	Whitelist allowed users

5.2.5 Firewall (UFW) Default policy: deny incoming, allow outgoing.

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp   # SSH
ufw allow 80/tcp   # HTTP
ufw allow 443/tcp  # HTTPS
ufw allow 8000/tcp # Coolify
ufw --force enable
```

Additional ports opened later for voice stack:

```
ufw allow 3100/tcp      # API (direct access)
ufw allow 5060/tcp      # SIP signaling (TCP)
ufw allow 5060/udp      # SIP signaling (UDP)
ufw allow 7880/tcp      # LiveKit API
ufw allow 7881/tcp      # LiveKit TCP fallback
ufw allow 10000:20000/udp  # RTP media
ufw allow 50000:60000/udp  # WebRTC ICE
```

5.2.6 Fail2Ban Configuration Creates `/etc/fail2ban/jail.local`:

Setting	Value	Purpose
`bantime`	1h	Default ban duration
`findtime`	10m	Window for counting failures
`maxretry`	5	Default max failures
`ignoreip`	127.0.0.1/8	Never ban localhost
SSH `maxretry`	3	SSH gets stricter limits
SSH `bantime`	24h	SSH bans last 24 hours

5.2.7 Swap Configuration Creates a 4 GB swap file (recommended for 8 GB RAM servers):

```
fallocate -l 4G /swapfile
chmod 600 /swapfile
mkswap /swapfile
swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
echo 'vm.swappiness=10' >> /etc/sysctl.conf  # Prefer RAM over swap
```

5.2.8 System Tuning

```
net.core.somaxconn = 65535          # Max socket backlog
net.ipv4.tcp_max_syn_backlog = 65535 # Max SYN queue
fs.file-max = 2097152               # Max open files
```

File descriptor limits for deploy user: 65535 soft/hard.

```
apt install -y unattended-upgrades
echo 'Unattended-Upgrade::Automatic-Reboot "false";' >> /etc/apt/apt.conf.d/50unattended-upgr
```

5.2.9 Automatic Security Updates Automatic reboot is disabled to prevent unexpected voice call interruptions.

5.3 SSH Key Management

Always use ed25519 keys:

```
# Generate key (local machine)
ssh-keygen -t ed25519 -C "your@email.com" -f ~/.ssh/id_ed25519

# Copy to server
ssh-copy-id -i ~/.ssh/id_ed25519 root@YOUR_IP

# Test connection
ssh -i ~/.ssh/id_ed25519 root@YOUR_IP
```

After the hardening script runs, also test the deploy user:

```
ssh -i ~/.ssh/id_ed25519 deploy@YOUR_IP
```

5.4 Hetzner Cloud Firewall

Create a firewall named `lumentra-voice` in the Hetzner Cloud Console with these inbound rules:

Protocol	Port Range	Source	Description
TCP	22	Any	SSH
TCP	80	Any	HTTP
TCP	443	Any	HTTPS
TCP	3100	Any	API (direct)
TCP+UDP	5060	Any	SIP signaling
TCP	7880-7881	Any	LiveKit API + TCP
TCP	8000	Any	Coolify
UDP	10000-20000	Any	RTP media
UDP	50000-60000	Any	WebRTC ICE

Both Hetzner firewall and UFW are active. A port must be open in both to be accessible. Hetzner firewall is the first layer (network-level), UFW is the second (host-level).

5.5 Enable SSH on Boot

Hetzner Ubuntu images sometimes have SSH disabled. Ensure it starts on boot:

```
systemctl enable ssh
systemctl start ssh
```

---

6. Coolify Deployment

6.1 Installing Coolify

File: `/hetzner/2-install-coolify.sh`

```
ssh root@YOUR_IP
curl -fsSL https://cdn.coollabs.io/coolify/install.sh | bash
```

Coolify installs Docker automatically. After installation:

1. Wait 30-60 seconds for Coolify to start.
2. Access at `http://YOUR_IP:8000`.
3. Create admin account.

4. **Enable 2FA** (Settings -> Security).

Add the deploy user to the docker group:

```
usermod -aG docker deploy
```

6.2 Coolify Initial Setup

1. **Sources -> Add -> GitHub App**: Connect your GitHub account via OAuth.
2. **Projects -> Add**: Create project named `Lumentra`.
3. **Server Settings -> Docker Cleanup**: Disable "Delete Unused Volumes" (prevents data loss).

6.3 Deploy the API

1. Projects -> Lumentra -> Add Resource -> **Application**.
2. Source: GitHub -> `callagent` repo.
3. Build Pack: **Dockerfile**.
4. Base Directory: `/lumentra-api`.
5. Port: `3100`.
6. Domain: `api.lumentraai.com`.
7. Enable SSL (Let's Encrypt).
8. Add all environment variables from Section 3 (database, LLM, voice, server).
9. Health Check Path: `/health`.
10. Health Check Interval: 30s.
11. Click **Deploy**.

Coolify UUID for API: `scog8ocs4884cos8gscw0kss`

6.4 Deploy the Dashboard

1. Add Resource -> **Application**.
2. Source: GitHub -> `callagent` repo.
3. Build Pack: **Dockerfile**.
4. Base Directory: `/lumentra-dashboard`.
5. Port: `3000`.
6. Domain: `app.lumentraai.com`.
7. Enable SSL.
8. Add dashboard environment variables (Section 3.9).
9. **Build Arguments** (set in Coolify's build args section):
   - `NEXT_PUBLIC_API_URL=https://api.lumentraai.com`
   - `NEXT_PUBLIC_APP_URL=https://app.lumentraai.com`
   - `NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co`
   - `NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJ...`
10. Click **Deploy**.

Coolify UUID for Dashboard: `hc44wc84swwo80s8k4gw88oo`

6.5 The Artisan Tinker Workaround

Coolify's API auth tokens are broken in some versions. Use the artisan tinker command to trigger deployments programmatically via SSH:

```
# Deploy the API
ssh root@YOUR_IP "docker exec coolify php artisan tinker --execute=\"\\\$app = App\Models\App

# Deploy the Dashboard
ssh root@YOUR_IP "docker exec coolify php artisan tinker --execute=\"\\\$app = App\Models\App
```

6.6 Auto-Deploy via Webhooks

1. In each Coolify application -> Settings -> Webhooks.
2. Copy the webhook URL.
3. In GitHub repo -> Settings -> Webhooks -> Add webhook:
    - Payload URL: (paste Coolify webhook URL)
    - Content type: `application/json`
    - Events: Just push events
    - Active: checked

Pushing to `main` now auto-deploys.

6.7 Coolify Network Architecture

Coolify creates its own Docker network. The API container is accessible at:

- `10.0.1.5:3100` – Coolify internal network IP
- `localhost:3100` – Host port mapping

The LiveKit agent (running outside Coolify) reaches the API via `http://10.0.1.5:3100`.

---

7. LiveKit Voice Stack Deployment

7.1 Overview

The LiveKit voice stack runs separately from Coolify, managed by a standalone `docker-compose.yml` at `/opt/livekit/`. This separation is necessary because:

1. LiveKit Server and SIP bridge require `network_mode: host`.
2. Coolify does not support host networking for managed applications.
3. The agent needs access to both LiveKit (host network) and the API (Coolify network).

7.2 LiveKit Server Configuration

File: `/lumentra-agent/livekit/livekit.yaml`

```yaml
port: 7880

keys:
  APIc4ecf671a4b0eab56ceb2cd4: 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc9703

rtc:
  tcp_port: 7881
  port_range_start: 50000
  port_range_end: 60000
  use_ice_lite: true

redis:
  address: localhost:6379

logging:
  level: info

room:
  auto_create: true
  empty_timeout: 300
```

Configuration Options Explained

Option	Value	Purpose
`port`	`7880`	HTTP API and WebSocket signaling port
`keys`	API key: secret pair	Authentication for agents and clients
`rtc.tcp_port`	`7881`	TCP fallback when UDP is blocked by firewalls
`rtc.port_range_start`	`50000`	Start of UDP port range for ICE candidates
`rtc.port_range_end`	`60000`	End of UDP port range (10,000 ports for concurrent calls)
`rtc.use_ice_lite`	`true`	Server-side ICE lite (recommended for self-hosted)
`redis.address`	`localhost:6379`	Redis for room state and coordination
`logging.level`	`info`	Set to debug for troubleshooting SIP/WebRTC issues
`room.auto_create`	`true`	Rooms are created automatically when agents/SIP join
`room.empty_timeout`	`300`	Clean up empty rooms after 5 minutes

Key generation:

```
# Generate a new API key/secret pair
openssl rand -hex 12   # API key prefix
openssl rand -hex 32   # API secret
```

7.3 SIP Configuration

File: `/lumentra-agent/livekit/sip.yaml`

```yaml
log_level: info

api_key: APIc4ecf671a4b0eab56ceb2cd4
api_secret: 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc970386
ws_url: ws://localhost:7880

redis:
  address: localhost:6379

sip_port: 5060
rtp_port: 10000-20000
```

Configuration Options Explained

Option	Value	Purpose
`log_level`	`info`	Logging verbosity (use debug for SIP troubleshooting)
`api_key`	LiveKit API key	Must match livekit.yaml keys
`api_secret`	LiveKit API secret	Must match livekit.yaml keys

Option	Value	Purpose
`ws_url`	`ws://localhost:7880`	Connection to LiveKit server (local, both on host net)
`redis`	`localhost:6379`	Must match LiveKit server Redis config
`sip_port`	`5060`	SIP signaling port (standard)
`rtp_port`	`10000-20000`	RTP media port range (10,000 ports for concurrent media)

7.4 SIP Trunk and Dispatch Rules

SIP trunks and dispatch rules are configured via the `lk` CLI tool, not in config files.

Current configuration:

Resource	ID	Details
Inbound Trunk	`ST_SUJtKjT9TEgv`	SignalWire inbound, +19458001233, krisp enabled
Dispatch Rule	`SDR_2bZEobprwRXq`	Routes to `lumentra-voice-agent`, room prefix: `call-`

```
# SSH to server, then:
lk --url http://localhost:7880 \
   --api-key APIc4ecf671a4b0eab56ceb2cd4 \
   --api-secret 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc970386 \
   sip inbound list

lk --url http://localhost:7880 \
   --api-key APIc4ecf671a4b0eab56ceb2cd4 \
   --api-secret 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc970386 \
   sip dispatch list
```

Listing SIP Resources

```
lk sip inbound create \
  --name "SignalWire Inbound" \
  --numbers "+19458001233" \
  --krisp-enabled
```

Creating a New Inbound Trunk

```
lk sip dispatch create \
  --name "Route to Voice Agent" \
  --room-prefix "call-" \
  --agent-name "lumentra-voice-agent"
```

Creating a Dispatch Rule

7.5 Production Deployment on Server

```
# On the server
mkdir -p /opt/livekit
cd /opt/livekit

# Create docker-compose.yml (copy from project or create manually)
# Create .env file with all required variables
# Copy livekit.yaml and sip.yaml to /opt/livekit/

# Start the stack
docker compose up -d

# Verify all services are running
docker compose ps
docker compose logs --tail 50

# Check LiveKit is responding
curl http://localhost:7880
```

Critical: Do NOT use ${VAR} interpolation in docker-compose files written over SSH. Hetzner's shell will expand them to empty strings. Use `env_file: .env` instead.

```
# CORRECT: Use env_file
agent:
  env_file: .env

# WRONG: Shell expands ${VAR} to empty over SSH
agent:
  environment:
    - OPENAI_API_KEY=${OPENAI_API_KEY}
```

7.6 Voice Agent Pipeline Details

The Python agent (`lumentra-agent/agent.py`) implements this pipeline:

1. **Process prewarm:** Loads Silero VAD model during process startup (not per-call).
2. **Job entry:** Waits for SIP participant, extracts `sip.trunkPhoneNumber` and `sip.phoneNumber`.
3. **Tenant lookup:** Calls `GET /internal/tenants/by-phone/:phone` to get config and system prompt.
4. **Session creation:** Configures STT, LLM, TTS, VAD, turn detection, noise cancellation.
5. **Agent start:** `LumentraAgent.on_enter()` sends the greeting message.
6. **Duration watchdog:** Enforces max call duration (default 15 min):
   - At T-2min: Nudges agent to wrap up.
   - At T-30s: Final warning, offers transfer.
   - At T-0: Force transfers to human or ends call.
7. **Call shutdown:** Logs call data, cancels watchdog, sends usage metrics.

```
stt=deepgram.STT(
    model="nova-3",          # Latest Deepgram model
    language="multi",         # Multi-language support
    smart_format=True,        # Punctuation, capitalization
    keyterm=[business_name],  # Boost business name recognition
)
```

STT Configuration

```
llm = openai.LLM(
    model="gpt-4.1-mini",    # Best balance of quality/speed/cost
    temperature=0.8,          # Creative but not chaotic
)
```

LLM Configuration

```
tts=cartesia.TTS(
    model="sonic-3",                                # Latest Cartesia model
    voice=tenant_config["voice_config"]["voice_id"], # Per-tenant voice
    speed=0.95,                                      # Slightly slower for clarity
    emotion=["Content"],                             # Pleasant tone
)
```

TTS Configuration

```
session = AgentSession(
    ...
    turn_detection=MultilingualModel(),
    preemptive_generation=True,         # Start generating before user finishes
    resume_false_interruption=True,     # Resume if user didn't actually interrupt
    false_interruption_timeout=1.5,     # 1.5s window for false interruption
    min_endpointing_delay=0.8,          # Min silence before considering turn complete
    max_endpointing_delay=2.5,          # Max silence before forcing turn complete
)
```

Turn Detection Tuning

Available Tools

Tool	API Endpoint	Purpose
`check_availability`	`POST /internal/voice-tools/check_availability`	Check open appointment slots
`create_booking`	`POST /internal/voice-tools/create_booking`	Book an appointment
`create_order`	`POST /internal/voice-tools/create_order`	Place a food order
`transfer_to_human`	`POST /internal/voice-tools/transfer_to_human`	SIP REFER transfer to human
`end_call`	`POST /internal/voice-tools/end_call`	Hang up the call
`log_note`	`POST /internal/voice-tools/log_note`	Save caller notes to CRM

---

8. SIP Trunk Setup (SignalWire)

8.1 SignalWire Account Creation

1. Go to https://signalwire.com and create an account.
2. Create a new Space (e.g., `your-company`).

3. Note your:
    - **Space URL:** `your-company.signalwire.com`
    - **Project ID:** Found in Settings -> API
    - **API Token:** Create one in Settings -> API -> API Tokens

8.2 Phone Number Purchase

1. In SignalWire Dashboard -> Phone Numbers -> Buy a Number.
2. Select your desired area code and number.
3. Note the full E.164 number (e.g., `+19458001233`).

8.3 Configure SIP Domain App

1. Go to Phone Numbers -> SIP Domain Apps.
2. Create a new SIP Domain App or find the existing one.
3. Note the SIP domain (e.g., `your-company.sip.signalwire.com`).

8.4 Configure the Phone Number for SIP

1. Go to Phone Numbers -> select your number.
2. Set **Call Handling** to **SIP Endpoint**.
3. Set the SIP endpoint to: `sip:+YOURNUMBER@YOUR_SERVER_IP:5060`
    - Or configure a SIP trunk pointing to your server IP.

8.5 Create LiveKit SIP Trunk

On your server, use the `lk` CLI:

```
# Install lk CLI if not present
curl -sSL https://get.livekit.io/cli | bash

# Create inbound SIP trunk
lk --url http://localhost:7880 \
   --api-key YOUR_API_KEY \
   --api-secret YOUR_API_SECRET \
   sip inbound create \
   --name "SignalWire Inbound" \
   --numbers "+19458001233" \
   --krisp-enabled
```

Note the Trunk ID (e.g., `ST_SUJtKjT9TEgv`).

8.6 Create Dispatch Rule

```
lk --url http://localhost:7880 \
   --api-key YOUR_API_KEY \
   --api-secret YOUR_API_SECRET \
   sip dispatch create \
   --name "Route to Voice Agent" \
   --room-prefix "call-" \
   --agent-name "lumentra-voice-agent"
```

Note the Dispatch Rule ID (e.g., `SDR_2bZEobprwRXq`).

8.7 Testing

1. Call the phone number from your cell phone.
2. Monitor agent logs:

```
cd /opt/livekit && docker compose logs agent -f
```

3. You should see:
   - "Call started: dialed=+19458001233 caller=+1YOURNUMBER room=call-xxxxx"
   - Tenant config fetched successfully
   - Agent greeting played
4. Check LiveKit rooms:

```
lk --url http://localhost:7880 \
   --api-key YOUR_API_KEY \
   --api-secret YOUR_API_SECRET \
   room list
```

8.8 Troubleshooting SIP

Symptom	Likely Cause	Fix
No SIP INVITE reaching server	Hetzner/UFW firewall blocking 5060	Open port 5060 TCP+UDP in both firewalls
SIP INVITE received but no audio	RTP ports blocked (10000-20000)	Open UDP 10000-20000 in both firewalls
"No trunk found" in SIP logs	Trunk number mismatch	Verify trunk number matches dialed number
"No dispatch rule" in SIP logs	Dispatch rule not created	Create dispatch rule with `lk` CLI
Agent not picking up	Agent not running or wrong agent name	Check agent logs, verify agent_name match

---

9. Nginx Configuration

9.1 Development Configuration

File: `/nginx.conf`

Used with `docker compose --profile production up -d`. Simple HTTP-only reverse proxy.

```
upstream api {
    server api:3100;     # Docker service name
}

upstream dashboard {
    server dashboard:3000;  # Docker service name
}
```

Upstream Definitions

```
limit_req_zone $binary_remote_addr zone=api_limit:10m rate=10r/s;
```

Rate Limiting

- **Zone size:** 10 MB (holds ~160,000 IP addresses).
- **Rate:** 10 requests per second per IP.

Location Blocks

Location	Upstream	Rate Limit	WebSocket	Timeout	Purpose
`/api/`	api	10r/s burst=20	Yes	Default	API routes
`/webhooks/`	api	None	No	Default	Webhook endpoints
`/signalwire/`	api	None	Yes	86400s	SignalWire media streams
`/health`	api	None	No	Default	Health check
`/`	dashboard	None	Yes	Default	Dashboard (catch-all)

9.2 Production Configuration

File: `/hetzner/nginx.conf`

Full production configuration with SSL, security headers, and optimized settings.

```
user www-data;
worker_processes auto;          # One worker per CPU core
events {
    worker_connections 2048;    # Max connections per worker
    multi_accept on;            # Accept multiple connections at once
    use epoll;                  # Linux-optimized event model
}
```

Global Settings

```
sendfile on;
tcp_nopush on;
tcp_nodelay on;
keepalive_timeout 65;
server_tokens off;             # Hide Nginx version
```

HTTP Settings

```
gzip on;
gzip_vary on;
gzip_proxied any;
gzip_comp_level 6;
gzip_types text/plain text/css text/xml application/json application/javascript
           application/rss+xml application/atom+xml image/svg+xml;
```

Gzip Compression

Rate Limiting Zones

Zone	Size	Rate	Purpose
`api_limit`	10m	30r/s	API endpoints
`general_limit`	10m	10r/s	Dashboard and general
`conn_limit`	10m	–	Connection count limiting

```
upstream lumentra_api {
    server 127.0.0.1:3100;
    keepalive 32;              # Keep 32 persistent connections
}

upstream lumentra_dashboard {
    server 127.0.0.1:3000;
    keepalive 32;
}
```

Upstream Configuration

HTTP Server (Port 80) Handles: - Certbot ACME challenges at `/.well-known/acme-challenge/`. - Redirects all other traffic to HTTPS with 301.

API HTTPS Server (api.lumentraai.com, Port 443)

Location	Rate Limit	Special Config	Purpose
`/signalwire/stream`	None	WebSocket upgrade, 86400s timeout	SIP media streams
`/signalwire/`	None	60s timeouts	SIP webhooks
`/health`	None	access_log off	Health checks
`/api/`	30r/s burst=50, 20 conn	60s timeouts	API routes
`/`	10r/s burst=20	Default timeouts	Catch-all

Dashboard HTTPS Server (lumentraai.com, Port 443)

Location	Rate Limit	Special Config	Purpose
`/`	10r/s burst=30, 30 conn	WebSocket upgrade	Next.js application
`/_next/static`	None	Cache 365 days, immutable	Static assets

```
add_header X-Frame-Options "SAMEORIGIN" always;
add_header X-Content-Type-Options "nosniff" always;
add_header X-XSS-Protection "1; mode=block" always;
add_header Referrer-Policy "strict-origin-when-cross-origin" always;
```

Security Headers (Applied to Both Servers)

```
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:...;
ssl_prefer_server_ciphers off;
ssl_session_timeout 1d;
ssl_session_cache shared:SSL:50m;
ssl_session_tickets off;
```

SSL Settings

9.3 Installing and Configuring Nginx

```
# Install Nginx
apt install -y nginx

# Copy production config
cp hetzner/nginx.conf /etc/nginx/nginx.conf

# Test configuration
nginx -t

# Reload
systemctl reload nginx

# Enable on boot
systemctl enable nginx
```

---

10. SSL/TLS Configuration

10.1 Certbot Installation

```
apt install -y certbot python3-certbot-nginx
```

10.2 Obtaining Certificates

```
# API subdomain
certbot --nginx -d api.lumentraai.com --non-interactive --agree-tos -m admin@lumentraai.com

# Dashboard/main domain
certbot --nginx -d lumentraai.com -d www.lumentraai.com --non-interactive --agree-tos -m admi

# Dashboard app subdomain
certbot --nginx -d app.lumentraai.com --non-interactive --agree-tos -m admin@lumentraai.com
```

Certificates are stored at `/etc/letsencrypt/live/DOMAIN/`.

10.3 Auto-Renewal

Certbot installs a systemd timer for automatic renewal:

```
# Verify timer is active
systemctl list-timers | grep certbot

# Test renewal (dry run)
```

```
certbot renew --dry-run

# Manual renewal
certbot renew
```

The timer runs twice daily and renews certificates expiring within 30 days.

10.4 Nginx SSL Integration

The production nginx.conf references certificates at:

```
ssl_certificate /etc/letsencrypt/live/api.lumentra.app/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/api.lumentra.app/privkey.pem;
```

Update these paths to match your domain.

10.5 Cloudflare SSL Mode

When using Cloudflare as a reverse proxy (orange cloud enabled):

1. Set SSL/TLS mode to **Full (strict)** in Cloudflare dashboard.
2. This means: Client -> Cloudflare (HTTPS) -> Your server (HTTPS with valid cert).
3. The Certbot certificate on your server must be valid (not self-signed).

10.6 Cloudflare Origin Certificates (Alternative)

Instead of Certbot, you can use Cloudflare Origin Certificates:

1. In Cloudflare -> SSL/TLS -> Origin Server -> Create Certificate.
2. Download the certificate and private key.
3. Install on your server:

   ```
   mkdir -p /etc/cloudflare/certs
   cp origin.pem /etc/cloudflare/certs/cert.pem
   cp origin-key.pem /etc/cloudflare/certs/key.pem
   chmod 600 /etc/cloudflare/certs/key.pem
   ```

4. Update nginx.conf to point to these files.

Origin certificates are valid for 15 years but only work with Cloudflare proxy enabled.

---

11. DNS Configuration

11.1 Add Domain to Cloudflare

1. Go to Cloudflare Dashboard.
2. Click “Add a Site” -> enter your domain.
3. Select Free plan.
4. Cloudflare scans existing DNS records.
5. Update nameservers at your domain registrar to Cloudflare’s.

11.2 DNS Records

Type	Name	Content	Proxy	Purpose
A	@	178.156.205.145	Proxied	Main site

Type	Name	Content	Proxy	Purpose
A	`app`	`178.156.205.145`	Proxied	Dashboard
A	`api`	`178.156.205.145`	Proxied	API
A	`www`	`178.156.205.145`	Proxied	www redirect

Important: Keep proxy ON (orange cloud) for DDoS protection and performance.

11.3 Cloudflare Settings

Setting	Value	Location
SSL/TLS mode	Full (strict)	SSL/TLS -> Overview
Always Use HTTPS	On	SSL/TLS -> Edge Certificates
Automatic HTTPS Rewrites	On	SSL/TLS -> Edge Certificates
WebSockets	On	Network
Minimum TLS Version	1.2	SSL/TLS -> Edge Certificates
HTTP/2	On	Speed -> Optimization

---

12. Database Setup and Migrations

12.1 Supabase Project Setup

1. Create a project at https://supabase.com.
2. Note your:
   - **Project URL:** `https://xxxx.supabase.co`
   - **Anon Key:** Settings -> API -> anon/public
   - **Service Role Key:** Settings -> API -> service_role (keep secret)
   - **Database URL:** Settings -> Database -> Connection string (URI)

12.2 Migration Reference

All migrations are in `/lumentra-api/migrations/`. Run them in order in the Supabase SQL Editor.

Migration	File	Creates/Modifies
001	`001_initial.sql`	`tenants`, `calls`, `bookings`, `callback_queue`, `sms_messages` tables, `update_updated_at()` trigger function, indexes

Migration	File	Creates/Modifies
002	002_crm_schema.sql	contacts, contact_notes, contact_activity, resources, availability_templates, availability_slots, notification_templates, notifications, notification_preferences, audit_logs tables. Adds contact_id to calls/bookings/sms/callbacks. RLS policies. Views: contact_summary, upcoming_bookings. Functions: normalize_phone(), update_contact_metrics(), generate_confirmation_code(), seed_notification_templates().
003	003_seed_test_data.sql	Seed data for testing (skip in production)
004	004_seed_simple.sql	Simple seed data (skip in production)
005	005_voicemails.sql	voicemails table with RLS
006	006_pizza_demo.sql	Demo tenant data (skip in production)
007a	007_llm_function_helpers.sql	LLM helper functions
007b	007_conversation_logs.sql	conversation_logs table, training_data_export view
008	008_hotel_demo.sql	Demo tenant data (skip in production)
009	009_auth_tenant_members.sql	tenant_members table, RLS for all tables based on membership, helper functions: get_user_tenants(), user_has_tenant_access(), get_user_tenant_role()
010	010_custom_instructions.sql	Adds custom_instructions and questionnaire_answers to tenants
011a	011_auth_user_trigger.sql	handle_new_user() trigger on auth.users
011b	011_responses.sql	Response templates table
012	012_fixes_and_improvements.sql	Adds contact_email, setup_completed to tenants. Fixes voicemails trigger. Service role bypass policies. handle_new_user() improvements. seed_tenant_defaults() auto-trigger.

Migration	File	Creates/Modifies
013	`013_setup_wizard_tables.sql`	`port_requests`, `tenant_capabilities`, `tenant_integrations`, `tenant_promotions`, `phone_configurations`, `escalation_contacts`, `pending_bookings` tables. Adds `setup_step`, `status`, `location_city`, `location_address`, `assisted_mode`, `after_hours_behavior`, `transfer_behavior` to tenants. Full RLS for all new tables.
014	`014_pending_bookings.sql`	`pending_bookings` table (may conflict with 013, run only if needed)
015	`015_vapi_phone_number_id.sql`	Adds `vapi_phone_number_id` to tenants (legacy Vapi support)
016	`016_sip_trunk_support.sql`	Adds `sip_uri`, `sip_username` to `phone_configurations`. Updates `setup_type` constraint to include `sip`.
017	`017_call_metadata_and_indexes.sql`	Adds `metadata` JSONB to calls. Composite indexes for dashboard COUNT queries.
018	`018_voice_pipeline_flag.sql`	Adds `voice_pipeline` column to tenants (`custom` or `livekit`)
019	`019_deals_and_tasks.sql`	`deals` and `tasks` tables for CRM pipeline

12.3 Running Migrations

For a fresh deployment, run all migrations in numeric order:

1. Open the Supabase SQL Editor (https://supabase.com/dashboard/project/YOUR_PROJECT/sql/new).
2. Paste each migration file content.
3. Click “Run” (or Ctrl+Enter).
4. Verify no errors.

Skip these migrations in production:

- `003_seed_test_data.sql` – Test data only.
- `004_seed_simple.sql` – Simple seed data only.
- `006_pizza_demo.sql` – Demo data only.
- `008_hotel_demo.sql` – Demo data only.

12.4 Migration Ordering Rules

1. Always run migrations in numeric order.
2. Migrations are forward-only – there are no down migrations.
3. All migrations use `IF NOT EXISTS` / `IF EXISTS` guards, so re-running is safe.
4. Some migration numbers have duplicates (e.g., two `007` files, two `011` files). Run all files regardless.

12.5 Rollback Procedures

Since migrations are forward-only, rollback requires manual SQL:

```sql
-- Example: Roll back migration 019 (deals and tasks)
DROP TABLE IF EXISTS tasks CASCADE;
DROP TABLE IF EXISTS deals CASCADE;

-- Example: Roll back a column addition
ALTER TABLE tenants DROP COLUMN IF EXISTS voice_pipeline;
```

Always back up before running migrations:

```bash
PGPASSWORD=YOUR_PW pg_dump \
  -h db.YOUR_PROJECT.supabase.co \
  -p 5432 -U postgres -d postgres \
  --no-owner --no-acl \
  | gzip > backup_before_migration_$(date +%Y%m%d).sql.gz
```

12.6 Writing New Migrations

1. Create a new file: `migrations/NNN_description.sql` where NNN is the next number.
2. Use `IF NOT EXISTS` / `IF EXISTS` guards on all DDL.
3. Enable RLS on all new tables.
4. Add `service_role` bypass policy.
5. Add `tenant_members`-based RLS policies if the table is tenant-scoped.
6. Add appropriate indexes.
7. Add `update_updated_at()` trigger for tables with `updated_at`.
8. Test locally with `docker-compose.local.yml` before applying to Supabase.

12.7 Schema Files

The `/lumentra-api/schema/` directory contains consolidated schema files for local development:

File	Purpose
`001-tables.sql`	All table definitions
`002-indexes.sql`	All index definitions
`003-functions.sql`	All function definitions

These files are automatically loaded into the local PostgreSQL container via `docker-entrypoint-initdb.d`.

---

13. Post-Deployment Verification

13.1 Health Check Endpoints

Endpoint	Expected Response	What It Checks
GET `https://api.lumentraai.com/health`	`{"status":"healthy","services":{"database":{"connected":true},...`	DB connection, Redis cache
GET `https://api.lumentraai.com/health/ping`	pong (plain text)	API process is alive (no DB check)
GET `https://app.lumentraai.com`	HTML page load (200)	Dashboard is serving

13.2 Verification Checklist

```
[ ] API health check returns 200 with "healthy" status
[ ] API /health/ping returns "pong"
[ ] Dashboard loads in browser
[ ] SSL certificates valid (green lock icon)
[ ] DNS resolves correctly for all subdomains
[ ] Cloudflare proxy is active (orange cloud)
[ ] WebSocket connections work (check browser DevTools)
[ ] LiveKit server responds: curl http://localhost:7880
[ ] SIP bridge is running: docker compose logs sip --tail 10
[ ] Agent is running and connected: docker compose logs agent --tail 10
[ ] Redis is healthy: docker exec redis redis-cli ping
[ ] Test phone call connects and agent responds
[ ] Agent tools work (check availability, create booking)
[ ] Call logging works (check calls table after test call)
[ ] Contact auto-creation works (check contacts table)
[ ] Dashboard shows the test call in call history
[ ] Backup cron is scheduled: crontab -l
[ ] Auto-deploy webhook fires on git push
```

13.3 Testing a Phone Call End-to-End

1. Call the business number from your cell phone.
2. Monitor agent logs in real-time:

   ```
   ssh root@YOUR_IP "cd /opt/livekit && docker compose logs agent -f"
   ```

3. Expected log flow:

   ```
   Call started: dialed=+19458001233 caller=+1XXXXXXXXXX room=call-xxxxx
   Using OpenAI gpt-4.1-mini
   Duration watchdog started: 900s limit
   Tool called: check_availability with args: {...}
   Call logged: call-xxxxx (120s, booking)
   ```

4. After hanging up, verify in the dashboard:
   - Call appears in call history.
   - Contact was auto-created.
   - Transcript is populated.
   - Post-call automation ran (deals/tasks).

---

14. Monitoring and Observability

14.1 Health Check Monitoring

The API exposes two health endpoints:

`GET /health` – Full health check (tests DB connection):

```
{
  "status": "healthy",
  "timestamp": "2026-03-02T12:00:00.000Z",
  "latency": "5ms",
  "services": {
    "database": {
      "connected": true,
```

```
      "latency": "3ms"
    },
    "tenantCache": {
      "size": 5,
      "hitRate": "94.2%"
    }
  },
  "config": {
    "voiceStack": "livekit-agents",
    "nodeEnv": "production"
  }
}
```

Returns 503 when unhealthy (DB disconnected).

`GET /health/ping` – Quick liveness check (no DB):

Returns pong with 200. Use this for load balancer health checks.

14.2 Uptime Kuma Setup

1. Deploy Uptime Kuma via Coolify (One-Click Service).
2. Create monitors:

Monitor Name	Type	URL/Host	Interval	Alert After
API Health	HTTP	`https://api.lumentoai.com/health`	60s	3 failures
API Ping	HTTP	`https://api.lumentoai.com/health/ping`	30s	2 failures
Dashboard	HTTP	`https://app.lumentoai.com`	60s	3 failures
LiveKit API	TCP	`YOUR_IP:7880`	60s	3 failures
SIP Port	TCP	`YOUR_IP:5060`	60s	3 failures
Redis	TCP	`127.0.0.1:6379`	30s	2 failures

3. Configure notification channels (Slack, email, SMS for critical alerts).

14.3 Log Monitoring

```
# API logs
docker logs $(docker ps -qf "name=api") --tail 100 -f

# Dashboard logs
docker logs $(docker ps -qf "name=dashboard") --tail 100 -f

# Agent logs
cd /opt/livekit && docker compose logs agent --tail 100 -f

# SIP bridge logs
cd /opt/livekit && docker compose logs sip --tail 100 -f

# LiveKit server logs
cd /opt/livekit && docker compose logs livekit --tail 100 -f
```

Docker Container Logs

Log Rotation All Docker services are configured with JSON file logging:

```yaml
logging:
  driver: "json-file"
  options:
    max-size: "50m"   # API: 50MB per file
    max-file: "3"      # Keep 3 files
```

Total log storage per service: - API: 150 MB (50 MB x 3) - Agent: 60 MB (20 MB x 3) - SIP: 60 MB (20 MB x 3) - LiveKit: 60 MB (20 MB x 3)

14.4 System Resource Monitoring

```bash
# Real-time resource usage
htop

# Docker container resource usage
docker stats --no-stream

# Disk usage
df -h
du -sh /var/lib/docker/

# Memory usage
free -h

# CPU load
uptime
```

14.5 Voice Call Metrics

The agent collects per-call metrics via LiveKit's `metrics.UsageCollector`:

```python
@session.on("metrics_collected")
def _on_metrics_collected(ev):
    metrics.log_metrics(ev.metrics)
    usage_collector.collect(ev.metrics)
```

On shutdown, `usage_collector.get_summary()` logs: - STT tokens consumed. - LLM tokens consumed (prompt + completion). - TTS characters synthesized. - Total latency breakdown.

These metrics appear in agent logs. For persistent storage, route them to a metrics backend.

14.6 Database Monitoring

```cpp
-- Active connections
SELECT count(*) FROM pg_stat_activity;

-- Long-running queries (>5 seconds)
SELECT pid, now() - pg_stat_activity.query_start AS duration, query
FROM pg_stat_activity
WHERE (now() - pg_stat_activity.query_start) > interval '5 seconds'
  AND state != 'idle';

-- Table sizes
SELECT schemaname, tablename, pg_size_pretty(pg_total_relation_size(schemaname || '.' || tabl
FROM pg_tables
WHERE schemaname = 'public'
```

```sql
ORDER BY pg_total_relation_size(schemaname || '.' || tablename) DESC;

-- Index usage
SELECT relname, idx_scan, seq_scan, n_live_tup
FROM pg_stat_user_tables
ORDER BY seq_scan DESC;
```

---

15. Backup and Recovery

15.1 Backup Script Setup

File: `/hetzner/3-backup-supabase.sh`

Run the setup script on the server:

```bash
scp hetzner/3-backup-supabase.sh root@YOUR_IP:/root/
ssh root@YOUR_IP "bash /root/3-backup-supabase.sh"
```

It will prompt for your Supabase database credentials and create:

Path	Purpose
`/opt/lumentra/.backup-config`	Database credentials (chmod 600)
`/opt/lumentra/run-backup.sh`	Backup execution script
`/opt/lumentra/backups/`	Backup storage directory

15.2 Backup Schedule

A cron job runs daily at 3 AM UTC:

```
0 3 * * * /opt/lumentra/run-backup.sh >> /opt/lumentra/backups/backup.log 2>&1
```

Verify: `crontab -l`

15.3 Backup Retention

The backup script keeps the last 7 daily backups:

```bash
ls -t $BACKUP_DIR/lumentra_*.sql.gz | tail -n +8 | xargs -r rm
```

15.4 Manual Backup

```bash
# Run a backup now
/opt/lumentra/run-backup.sh

# Check backups
ls -la /opt/lumentra/backups/

# Check backup log
tail -f /opt/lumentra/backups/backup.log
```

15.5 Backup Command Details

The backup uses pg_dump with these options:

```
PGPASSWORD=$DB_PASS pg_dump \
  -h $DB_HOST \
  -p 5432 \
  -U postgres \
  -d postgres \
  --no-owner \       # Don't include ownership commands
  --no-acl \         # Don't include permission grants
  | gzip > $BACKUP_FILE
```

Typical backup size: 1-5 MB compressed for a small-medium database.

15.6 Restoring from Backup

```
# Decompress
gunzip lumentra_2026-03-01_03-00-00.sql.gz

# Restore to Supabase (CAUTION: this overwrites existing data)
PGPASSWORD=$DB_PASS psql \
  -h db.YOUR_PROJECT.supabase.co \
  -p 5432 \
  -U postgres \
  -d postgres \
  < lumentra_2026-03-01_03-00-00.sql
```

For selective restoration:

```
# Restore only specific tables
PGPASSWORD=$DB_PASS pg_restore \
  -h db.YOUR_PROJECT.supabase.co \
  -p 5432 \
  -U postgres \
  -d postgres \
  --data-only \
  --table=contacts \
  backup.dump
```

15.7 Disaster Recovery Plan

Scenario	Recovery Procedure	RTO
API service crash	Auto-restart via `unless-stopped` policy	<30s
Database connection lost	Auto-reconnect; health check shows "degraded"	<5min
Server completely down	Provision new server, restore from backup	1-2hrs
Supabase outage	Wait for Supabase recovery; data is in their backups	Varies
Corrupted database	Restore from last good pg_dump backup	30min
Lost LiveKit configuration	Recreate SIP trunk and dispatch rules via `lk` CLI	15min
Docker volumes deleted	Restore Redis data (or it rebuilds), restart services	10min

15.8 File Backup

In addition to database backups, back up these critical files:

```
# LiveKit configuration
tar czf /opt/lumentra/backups/livekit_config_$(date +%Y%m%d).tar.gz \
  /opt/livekit/docker-compose.yml \
  /opt/livekit/.env \
  /opt/livekit/livekit.yaml \
  /opt/livekit/sip.yaml

# Nginx configuration
tar czf /opt/lumentra/backups/nginx_config_$(date +%Y%m%d).tar.gz \
  /etc/nginx/nginx.conf

# SSL certificates
tar czf /opt/lumentra/backups/ssl_certs_$(date +%Y%m%d).tar.gz \
  /etc/letsencrypt/
```

---

16. Update Procedures

16.1 Updating the API

Via Coolify Auto-Deploy Push to `main` and Coolify auto-deploys:

```
git add .
git commit -m "Fix: description of change"
git push origin main
```

Via Manual Coolify Deploy

1. Open Coolify at `http://YOUR_IP:8000`.
2. Go to Projects -> Lumentra -> API.
3. Click “Deploy”.

```
ssh root@YOUR_IP "docker exec coolify php artisan tinker --execute=\"\\\$app = App\Models\App
```

Via Artisan Tinker (Programmatic)

16.2 Updating the Dashboard

Same as API but uses the dashboard UUID:

```
# Force rebuild (needed when NEXT_PUBLIC_* vars change)
ssh root@YOUR_IP "docker exec coolify php artisan tinker --execute=\"\\\$app = App\Models\App
```

Note: Dashboard builds are slower (~2-3 minutes) because Next.js must compile the entire application.

16.3 Updating the Voice Agent

The agent runs outside Coolify, so update manually:

```
ssh root@YOUR_IP << 'EOF'
cd /opt/livekit
```

```
# Pull latest code
git pull origin main

# Rebuild the agent image
docker compose build agent

# Restart the agent (brief interruption to active calls)
docker compose up -d agent

# Verify
docker compose logs agent --tail 20
EOF
```

16.4 Updating LiveKit Server

```
ssh root@YOUR_IP << 'EOF'
cd /opt/livekit

# Update image version in docker-compose.yml
# Change: image: livekit/livekit-server:v1.8
# To:     image: livekit/livekit-server:v1.9

# Pull new image
docker compose pull livekit

# Restart LiveKit (active calls will be disconnected)
docker compose up -d livekit

# Verify
docker compose logs livekit --tail 20
curl http://localhost:7880
EOF
```

16.5 Updating LiveKit SIP Bridge

```
ssh root@YOUR_IP << 'EOF'
cd /opt/livekit

# Update image version
docker compose pull sip

# Restart SIP (active SIP calls will be disconnected)
docker compose up -d sip

# Verify
docker compose logs sip --tail 20
EOF
```

16.6 Database Migrations During Updates

1. **Always back up first:**

   ```
   ssh root@YOUR_IP "/opt/lumentra/run-backup.sh"
   ```

2. **Run the new migration in Supabase SQL Editor.**

3. **Deploy the API** (which expects the new schema).
4. **Never deploy API before running its required migration** – the API will crash on missing columns/tables.

16.7 Zero-Downtime Considerations

Component	Downtime During Update	Mitigation
API	~5-10 seconds	Coolify performs rolling restart
Dashboard	~10-30 seconds	Build takes time; old container serves until new is ready
Agent	~5 seconds	Active calls are interrupted
LiveKit	Active calls dropped	Schedule during low-traffic hours
SIP	Active calls dropped	Schedule during low-traffic hours
Database	None (DDL is fast)	Supabase applies migrations instantly

For true zero-downtime voice stack updates, run two agents simultaneously:

```
# Scale to 2 agents temporarily
docker compose up -d --scale agent=2

# Wait for new agent to be ready
sleep 30

# Scale back to 1 (old agent finishes current calls)
docker compose up -d --scale agent=1
```

---

17. Scaling Guide

17.1 Current Capacity (Single CCX13)

Metric	Estimated Capacity
Concurrent voice calls	20-30
API requests per second	500+
Dashboard concurrent users	100+
Database connections	100 (Supabase free tier)

17.2 Horizontal Scaling: API

The API is stateless and can scale horizontally:

1. **Multiple API instances behind a load balancer:**

```
upstream lumentra_api {
    server 10.0.1.5:3100;
    server 10.0.1.6:3100;
    keepalive 32;
}
```

2. **Health check routing:**

```
upstream lumentra_api {
    server 10.0.1.5:3100 max_fails=3 fail_timeout=30s;
    server 10.0.1.6:3100 max_fails=3 fail_timeout=30s;
}
```

3. **Sticky sessions** are NOT needed – the API is stateless.

17.3 Scaling: Voice Agent

```
# Scale to 3 agent instances
docker compose up -d --scale agent=3
```

Each agent instance can handle 10-15 concurrent calls. LiveKit distributes jobs across available agents automatically.

17.4 Scaling: LiveKit Server

LiveKit supports multi-node clustering via Redis:

1. Add more LiveKit server instances on different machines.
2. Point them all to the same Redis instance.
3. LiveKit automatically distributes rooms across nodes.

17.5 Database Connection Pooling

For high-traffic deployments, use Supabase's built-in PgBouncer:

- **Connection string:** Use the "Pooler" connection string from Supabase Settings -> Database.
- **Mode:** Transaction mode (recommended for serverless/short-lived connections).
- **Pool size:** 100 connections (Supabase Pro plan).

17.6 Scaling: Redis

Redis is used only for LiveKit room coordination. A single Redis instance handles thousands of rooms. For HA:

1. Use Redis Sentinel for automatic failover.
2. Or use a managed Redis (e.g., AWS ElastiCache, Upstash).

17.7 Scaling Thresholds

Metric	Threshold	Action
CPU usage >80% sustained	5 minutes	Add CPU or scale horizontally
Memory usage >85%	5 minutes	Increase memory or add instance
Disk usage >80%	–	Clean logs, expand disk
API response time >500ms (p95)	5 minutes	Scale API instances
Concurrent calls >25	–	Scale agent instances
Database connections >80	–	Enable connection pooling

---

18. Troubleshooting

18.1 API Issues

API Health Check Returns 503 **Error:** `{"status":"degraded","services":{"database":{"connected":f`

Causes and fixes:

1. **Supabase is down:** Check status.supabase.com. Wait for recovery.
2. **Wrong SUPABASE_URL or keys:** Verify environment variables match Supabase dashboard.
3. **Network issue:** Test from server: `curl -s https://YOUR_PROJECT.supabase.co/rest/v1/ -H "apikey: YOUR_ANON_KEY"`.

API Container Won't Start **Error in logs:** `Error: Cannot find module './dist/index.js'`

Fix: Build failed. Check build logs in Coolify. Common causes: - TypeScript compilation errors. - Missing npm dependencies. - Force rebuild: Set `CACHE_BUST` build arg to a new value.

500 Error on API Endpoints **Common cause:** Missing database migration.

Fix: Check which migration is needed by reading the error message. Run the migration in Supabase SQL Editor.

18.2 Dashboard Issues

Dashboard Shows Blank Page **Causes:**

1. **NEXT_PUBLIC_API_URL incorrect at build time:** Rebuild with correct value.
2. **API is down:** Check API health first.
3. **CORS issue:** Check browser DevTools -> Console for CORS errors.

"Invalid API Key" on Dashboard **Fix:** Verify `NEXT_PUBLIC_SUPABASE_ANON_KEY` matches Supabase dashboard. Requires rebuild.

18.3 Voice Call Issues

Calls Ring But No Agent Answers **Causes:**

1. **Agent not running:**

```
cd /opt/livekit && docker compose ps agent
docker compose logs agent --tail 50
```

2. **Agent name mismatch:** The dispatch rule must specify `agent_name: "lumentra-voice-agent"` which must match the `@server.rtc_session(agent_name="lumentra-voice-agent")` decorator.
3. **Agent cannot connect to LiveKit:** Check `LIVEKIT_URL` in agent environment.

Agent Says Nothing (No TTS) **Causes:**

1. **Invalid Cartesia API key:** Check `CARTESIA_API_KEY`.
2. **Invalid voice ID:** Check tenant's `voice_config.voice_id` in database.
3. **TTS error in logs:** `docker compose logs agent | grep -i "cartesia\|tts\|error"`.

Agent Cannot Hear Caller (No STT) **Causes:**

1. **Invalid Deepgram API key:** Check `DEEPGRAM_API_KEY`.
2. **No RTP audio reaching server:** Open UDP 10000-20000 in Hetzner firewall AND UFW.
3. **SIP bridge not forwarding audio:** Check SIP logs for RTP errors.

"No Tenant Found" Error **Error in agent logs:** `No tenant found for phone: +19458001233`

Causes:

1. **Phone number not in database:** Check tenants table for matching `phone_number`.
2. **Tenant is inactive:** Check `is_active = true` in tenants table.
3. **API internal endpoint unreachable:** Check `INTERNAL_API_URL` and `INTERNAL_API_KEY`.

Call Quality Issues (Choppy Audio, Delays) **Causes:**

1. **Network jitter:** Run `mtr YOUR_IP` to check network quality.
2. **Server CPU overloaded:** Check `htop` during a call.
3. **Endpointing too aggressive:** Increase `min_endpointing_delay` in agent.py.
4. **Missing noise cancellation:** Ensure `noise_cancellation.BVCTelephony()` is configured.

18.4 SIP Issues

```
# Check if port 5060 is open
nc -zvu YOUR_IP 5060

# Check UFW status
ufw status | grep 5060

# Check Hetzner firewall
~/.local/bin/hcloud firewall describe lumentra-voice
```

No SIP INVITE Reaching Server

SIP 404 Not Found The SIP bridge cannot find a matching trunk.

```
# List inbound trunks
lk --url http://localhost:7880 \
   --api-key YOUR_KEY \
   --api-secret YOUR_SECRET \
   sip inbound list
```

Verify the trunk's `numbers` field matches the dialed number.

SIP 500 Internal Server Error Check SIP bridge logs:

```
cd /opt/livekit && docker compose logs sip --tail 100
```

Common causes: Redis connection failed, LiveKit server unreachable.

18.5 SSL Issues

"ERR_SSL_PROTOCOL_ERROR" **Cause:** Cloudflare SSL mode mismatch.

Fix: Set Cloudflare SSL/TLS to "Full (strict)" and ensure Certbot certificate is valid.

```
# Test renewal
certbot renew --dry-run

# Force renewal
certbot renew --force-renewal
```

```
# Check certificate expiry
openssl x509 -dates -noout -in /etc/letsencrypt/live/YOUR_DOMAIN/cert.pem
```

Certificate Renewal Failed

18.6 Coolify Issues

Deploy Stuck in "Building"

1. Check build logs in Coolify UI.
2. SSH to server and check Docker:

   ```
   docker ps -a | grep build
   docker logs CONTAINER_ID
   ```

3. Common fix: restart Coolify:

   ```
   docker restart coolify
   ```

"Coolify API Token Not Working" Known issue in some Coolify versions. Use the artisan tinker workaround (Section 6.5).

18.7 Database Issues

RLS Policy Blocks API Queries The API uses `SUPABASE_SERVICE_ROLE_KEY` which bypasses RLS. If queries fail:

1. Verify the key is correct.
2. Ensure `service_role` policies exist: `CREATE POLICY service_role_xxx ON table FOR ALL TO service_role USING (true) WITH CHECK (true);`.

Slow Dashboard Queries Check for missing indexes:

```
-- Find sequential scans (missing indexes)
SELECT relname, seq_scan, idx_scan
FROM pg_stat_user_tables
WHERE seq_scan > 100
ORDER BY seq_scan DESC;
```

Migration 017 adds composite indexes for common dashboard COUNT queries.

18.8 Docker Issues

```
# Check disk usage
df -h

# Clean Docker
docker system prune -a --volumes

# Clean old images
docker image prune -a

# Clean old logs
truncate -s 0 /var/lib/docker/containers/*/*-json.log
```

"No Space Left on Device"

```
# Check exit code
docker inspect CONTAINER_ID | grep "ExitCode"

# Common exit codes:
# 0   - Normal exit
# 1   - Application error
# 137 - OOM killed (need more memory)
# 143 - SIGTERM (graceful shutdown)

# Check if OOM killed
docker inspect CONTAINER_ID | grep "OOMKilled"
dmesg | grep -i "oom\|killed"
```

Container Keeps Restarting

---

19. Security Checklist

19.1 Server Security

```
[ ] SSH hardening applied (key-only, no password auth)
[ ] Deploy user created with sudo access
[ ] Root login restricted to key-only
[ ] UFW firewall enabled with minimal open ports
[ ] Fail2Ban configured for SSH
[ ] Automatic security updates enabled
[ ] Swap configured (prevents OOM-related security issues)
[ ] File descriptor limits increased
```

19.2 Network Security

```
[ ] Hetzner cloud firewall active
[ ] UFW firewall active
[ ] Redis bound to localhost only (127.0.0.1)
[ ] PostgreSQL not directly exposed (Supabase managed)
[ ] Coolify port (8000) restricted to admin IPs (if possible)
[ ] HTTPS enforced on all public endpoints
[ ] Cloudflare proxy enabled (DDoS protection)
```

19.3 Application Security

```
[ ] All API keys stored as environment variables (not in code)
[ ] INTERNAL_API_KEY is a strong random secret (64+ chars)
[ ] ENCRYPTION_KEY is a strong random secret (32+ chars)
[ ] Supabase SERVICE_ROLE_KEY never exposed to client
[ ] RLS enabled on all database tables
[ ] Service role bypass policies only for backend
[ ] Rate limiting configured on API endpoints
[ ] Security headers set (X-Frame-Options, CSP, etc.)
[ ] Docker containers run as non-root users
[ ] No secrets in docker-compose.yml (use .env files)
```

19.4 Credentials Security

```
[ ] No credentials committed to git repository
```

```
[ ] .env files in .gitignore
[ ] API keys rotated every 90 days
[ ] Backup config file permissions set to 600
[ ] SSH keys use ed25519 (not RSA)
[ ] 2FA enabled on Coolify admin account
[ ] 2FA enabled on Supabase account
[ ] 2FA enabled on SignalWire account
```

19.5 Generating Secure Secrets

```bash
# 64-character hex secret (for INTERNAL_API_KEY, ENCRYPTION_KEY)
openssl rand -hex 32

# 32-character base64 secret
openssl rand -base64 32

# UUID
python3 -c "import uuid; print(uuid.uuid4())"
```

---

20. Cost Estimates

20.1 Monthly Infrastructure Costs

Service	Plan/Tier	Monthly Cost (USD)
Hetzner CCX13 server	4 vCPU, 8 GB RAM	~$16
Hetzner Volume (10 GB)	SSD	~$1
Supabase	Free tier	$0
Cloudflare	Free tier	$0
Domain registration	Annual	~$1/month
Infrastructure total		**~$18/month**

20.2 Per-Call Voice Costs

Service	Cost Per Unit	Per 5-min Call
Deepgram STT	$0.0043/min (nova-3)	~$0.022
OpenAI LLM	GPT-4.1-mini tokens	~$0.005
Cartesia TTS	$0.010/1000 chars	~$0.003
SignalWire SIP	$0.005/min	~$0.025
Total per call		**~$0.055**

With 88% prompt cache hits on GPT-4.1-mini, effective LLM cost drops to ~$0.001/call.

20.3 Monthly Volume Estimates

Volume	100 calls/month	1,000 calls/month	10,000 calls/month
Voice costs	$5.50	$55	$550
Infrastructure	$18	$18	$50 (scaled)
Total	**$23.50**	**$73**	**$600**

---

21. Quick Command Reference

21.1 SSH Access

```
# SSH to server (root)
ssh -i ~/.ssh/id_ed25519 root@178.156.205.145

# SSH to server (deploy user)
ssh -i ~/.ssh/id_ed25519 deploy@178.156.205.145
```

21.2 Container Management

```
# List all containers
docker ps --format 'table {{.Names}}\t{{.Status}}\t{{.Ports}}'

# View API logs
docker logs $(docker ps -qf "name=api") --tail 100 -f

# View dashboard logs
docker logs $(docker ps -qf "name=dashboard") --tail 100 -f

# Restart a specific container
docker restart CONTAINER_NAME
```

21.3 LiveKit Stack

```
# LiveKit service status
cd /opt/livekit && docker compose ps

# Agent logs
cd /opt/livekit && docker compose logs agent --tail 50 -f

# SIP logs
cd /opt/livekit && docker compose logs sip --tail 50 -f

# LiveKit server logs
cd /opt/livekit && docker compose logs livekit --tail 50 -f

# Restart all LiveKit services
cd /opt/livekit && docker compose restart

# Rebuild and restart agent only
cd /opt/livekit && docker compose build agent && docker compose up -d agent
```

21.4 LiveKit CLI

```
# List SIP trunks
lk --url http://localhost:7880 \
   --api-key APIc4ecf671a4b0eab56ceb2cd4 \
   --api-secret 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc970386 \
   sip inbound list

# List SIP dispatch rules
lk --url http://localhost:7880 \
```

```
   --api-key APIc4ecf671a4b0eab56ceb2cd4 \
   --api-secret 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc970386 \
   sip dispatch list

# List active rooms
lk --url http://localhost:7880 \
   --api-key APIc4ecf671a4b0eab56ceb2cd4 \
   --api-secret 7b284f366c10a9f0319a29352402a300a01362e960a84c492baaaeedfc970386 \
   room list
```

21.5 Backup and Database

```
# Manual backup
/opt/lumentra/run-backup.sh

# View backups
ls -la /opt/lumentra/backups/

# View backup log
tail -f /opt/lumentra/backups/backup.log

# Check cron jobs
crontab -l
```

21.6 Coolify Deployment

```
# Deploy API via artisan tinker
ssh root@178.156.205.145 "docker exec coolify php artisan tinker --execute=\"\\\$app = App\Mo

# Deploy Dashboard via artisan tinker
ssh root@178.156.205.145 "docker exec coolify php artisan tinker --execute=\"\\\$app = App\Mo

# Restart Coolify itself
ssh root@178.156.205.145 "docker restart coolify"
```

21.7 SSL and Nginx

```
# Test nginx config
nginx -t

# Reload nginx (no downtime)
systemctl reload nginx

# Renew SSL certificates
certbot renew

# Check certificate expiry
openssl x509 -dates -noout -in /etc/letsencrypt/live/YOUR_DOMAIN/cert.pem
```

21.8 System Monitoring

```
# Real-time system monitor
htop
```

```
# Docker resource usage
docker stats --no-stream

# Disk usage
df -h

# Memory usage
free -h

# Network connections
ss -tulpn

# Check firewall status
ufw status verbose

# Check Fail2Ban status
fail2ban-client status sshd

# System uptime and load
uptime
```

21.9 Hetzner Cloud CLI

```
# List servers
~/.local/bin/hcloud server list

# SSH via hcloud
~/.local/bin/hcloud server ssh Lumentra-ubuntu

# Describe firewall
~/.local/bin/hcloud firewall describe lumentra-voice

# Server power management
~/.local/bin/hcloud server reboot Lumentra-ubuntu
~/.local/bin/hcloud server poweroff Lumentra-ubuntu
~/.local/bin/hcloud server poweron Lumentra-ubuntu
```